

Agentic AI — A Beginner's Guide (and a Resume Project)

Every concept explained in plain language, with examples — then a project you can build and put on your resume

How to read this guide

You told me you're new to these terms, so this guide assumes **zero prior AI knowledge**. Every concept follows the same shape:

- **In one sentence** — the simplest possible definition.
- **The analogy** — a everyday comparison so it sticks.
- **How it actually works** — the real mechanics, still in plain words.
- **Example** — a concrete, worked scenario.
- **Why it matters for agents** — how this piece fits the bigger picture.

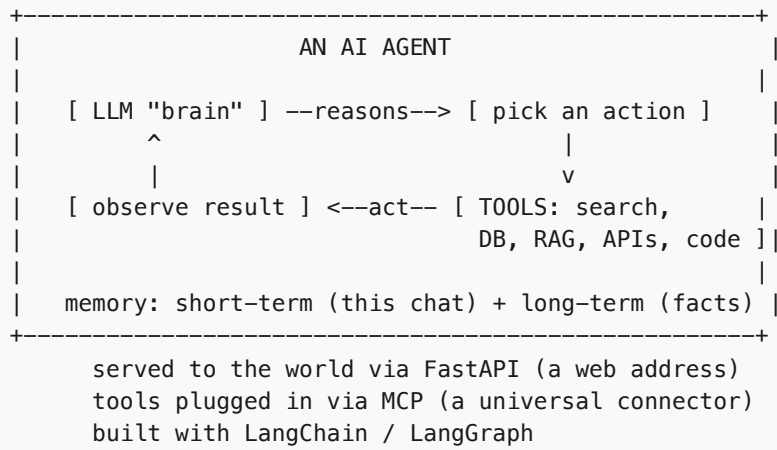
Read Part 1 top-to-bottom; each concept builds on the previous one. Part 2 then assembles everything into a single project you can build and describe in interviews.

A note on freshness (important). This field moves fast. Two things to know as of early 2026: (1) **MCP** (Model Context Protocol) was created by Anthropic in Nov 2024, but in Dec 2025 its governance was donated to the **Agentic AI Foundation** (a Linux-Foundation fund), so it's now an industry standard rather than an Anthropic-only project. (2) Library APIs (LangChain, LangGraph) change often — the *concepts* here are stable, but always check current docs for exact function names before you code.

The 30-second big picture

Before the details, here's the whole story in one paragraph so the pieces have somewhere to land:

A **Large Language Model (LLM)** is a program that predicts text. On its own it can only *talk*. An **AI agent** wraps an LLM in a **loop** and gives it **tools** (search, a calculator, a database, APIs) so it can *act* — decide what to do, do it, look at the result, and decide again, until a goal is met. **RAG** feeds the agent relevant facts from your own documents so it doesn't make things up. **MCP** is a universal plug so any agent can connect to any tool. **LangChain/LangGraph** are the toolkits you use to build the agent and its loop. **FastAPI** is how you put the finished agent behind a web address so other apps (or a chat UI) can use it. **Memory** lets the agent remember the current conversation (short-term) and things across sessions (long-term). **Multi-agent** means several specialized agents working together.



Everything below is just this picture, one box at a time.

Part 1A — The Foundations (LLM, Tokens, Embeddings, RAG)

1. What is an LLM (Large Language Model)?

In one sentence. An LLM is a very large computer program that has read enormous amounts of text and learned to **predict the next word**, over and over, to produce human-like writing.

The analogy. Think of the world's most well-read autocomplete. Your phone suggests the next word after "I'll be there in a..." ("minute"). An LLM does the same thing, but so well — across paragraphs, code, and reasoning — that the result feels like thinking.

How it actually works (plain words). - It was **trained** by being shown trillions of words and asked, again and again: "given this text so far, what word comes next?" Each time it guessed wrong, its internal settings ("weights" — billions of tunable numbers) were nudged. After enough nudging, it got very good at prediction. - At **use time**, you give it some text (the "prompt"), and it generates a reply **one token at a time** (see tokens below), each token chosen based on everything before it. - Crucially, it has **no memory between calls** on its own, and **no live access** to the internet or your files. It only knows what it learned during training plus whatever you put in the prompt right now. (This limitation is *exactly* why agents, RAG, and memory exist.)

Example.

```
You type: "Explain gravity to a 5-year-old."
The LLM generates, token by token:
  "Gravity" -> "is" -> "the" -> "force" -> "that" -> "pulls" -> ...
Result: "Gravity is the force that pulls things down toward the ground..."
```

Why it matters for agents. The LLM is the **"brain"** of an agent — it does the reasoning and decides what to do. But a brain with no hands (tools) and no notebook (memory) can only talk. The rest of this guide is about giving it hands and a notebook.

Popular LLMs you'll hear about: Anthropic's **Claude**, OpenAI's **GPT** family, Google's **Gemini**, Meta's open **Llama** models.

2. Tokens & Tokenization

In one sentence. A token is a small chunk of text (roughly a word or part of a word), and tokenization is the act of chopping text into those chunks — because LLMs read and write in tokens, not letters or whole words.

The analogy. LEGO. You don't build with a finished house; you build with bricks. Tokens are the LLM's bricks. Words get snapped apart into standard reusable pieces.

How it actually works. - A common algorithm is **BPE (Byte Pair Encoding)** — used by Llama, Gemma, Qwen and others. It learns the most frequent character-chunks in a language and uses those as tokens. - Common words are usually **one token** ("the", "cat"). Rare or long words split into **several** ("tokenization" might become "token" + "ization"). - Rough rule of thumb for English: **1 token $\approx$ 4 characters $\approx$ ¾ of a word**. So 100 tokens $\approx$ 75 words.

Example.

```
Text:    "Agentic AI is powerful."
Tokens:  ["Ag", "entic", " AI", " is", " powerful", "."]  (6 tokens)
        (notice the leading spaces are part of the token)
```

Why it matters (this is very practical). - **Context window:** every model can only "see" a limited number of tokens at once (e.g., tens or hundreds of thousands). Your prompt + the documents you attach + the reply all count against this budget. Run out, and older text must be dropped or summarized — this is the whole reason "memory" (Section 9) is a real engineering problem. - **Cost & speed:** you're billed per token (in and out), and more tokens = slower replies. Efficient agents keep token use lean.

3. Embeddings (turning meaning into numbers)

In one sentence. An embedding is a list of numbers that represents the *meaning* of a piece of text, so that a computer can measure how similar two texts are.

The analogy. Imagine placing every sentence as a pin on a giant map, where things with similar meaning sit close together. "How do I reset my password?" and "I forgot my login" would be near-neighbors, even though they share almost no words. An embedding is the **map coordinate** (just a long list of numbers, e.g. 1,536 of them) for a piece of text.

How it actually works. - A special model (an "embedding model") reads text and outputs a fixed-length list of numbers — a **vector**. - Texts with similar meaning get vectors that point in **similar directions**. We measure "similarity" with math (commonly **cosine similarity** — how close the angle between two vectors is).

Example.

```
"dog"    -> [0.91, 0.10, 0.44, ...]  \  these two vectors point
"puppy"  -> [0.88, 0.14, 0.41, ...]  /  in almost the same direction -> similar
"invoice"-> [0.02, 0.77, 0.13, ...]   \  points elsewhere -> not similar
```

Why it matters for agents. Embeddings are the secret behind **search by meaning** instead of search by keyword. That's the engine of RAG (next section) and of long-term memory (Section 9). A **vector database** is simply a database built to store these vectors and find the nearest ones fast.

4. Transformers & Attention (the engine, at a high level)

In one sentence. The transformer is the neural-network design that powers modern LLMs, and "attention" is its key trick: when processing a word, it looks at **all the other words** and decides which

ones matter most.

The analogy. Reading the sentence "The trophy didn't fit in the suitcase because **it** was too big." To know what "it" means, you glance back at "trophy" (not "suitcase"). Attention is the model doing exactly that glancing — weighing which earlier words are relevant to the word it's currently handling.

How it actually works (just the intuition — you don't need the math for a resume project). - Text is turned into tokens, then into vectors. - The **attention mechanism** lets every token "look at" every other token and pull in the context it needs. "Bank" near "river" vs "bank" near "money" ends up with different meaning because it attended to different neighbors. - Stacking many attention layers lets the model build up rich understanding. "Transformer" is the name of this whole stacked architecture (from the 2017 paper *Attention Is All You Need*).

Why it matters for agents. You rarely touch this directly — but when someone asks "how do LLMs actually work?" the honest one-liner is: *"They're transformer neural networks that use attention to weigh context, trained to predict the next token."* That single sentence signals you understand the engine, not just the buzzwords.

5. RAG (Retrieval-Augmented Generation)

In one sentence. RAG means: before the LLM answers, first **fetch relevant facts** from your own documents and paste them into the prompt — so the model answers from real, current, private information instead of guessing.

The analogy. An open-book exam. The LLM is a smart student, but it hasn't read *your* company's handbook. RAG hands it the exact right pages right before it answers the question.

The problem it solves. LLMs (a) don't know your private/internal documents, (b) have a training cutoff so they don't know recent events, and (c) will sometimes **"hallucinate"** — state something false with confidence. RAG fixes all three by grounding the answer in retrieved text.

How it actually works — two phases:

```
PHASE 1 – INDEXING (done once, ahead of time)
  your docs -> split into chunks -> embed each chunk -> store vectors
                                                    in a vector DB

PHASE 2 – RETRIEVAL + GENERATION (done per question)
  user question -> embed it -> find the most similar chunks in the
  vector DB (semantic search) -> paste those chunks into the prompt
  -> LLM writes an answer grounded in them (often with citations)
```

Example.

Question: "What's our refund window for enterprise plans?"

1. Embed the question.
2. Vector DB returns the 3 most similar chunks from your policy docs,
e.g. a paragraph: "Enterprise refunds are accepted within 30 days..."
3. Prompt to LLM becomes:
"Using ONLY the context below, answer the question.
Context: <the 30-day refund paragraph>
Question: What's our refund window for enterprise plans?"
4. LLM: "Enterprise plans can be refunded within 30 days of purchase."
(grounded in your real doc, not invented)

Why it matters for agents. RAG is often the agent's most important **tool**: "look something up in the knowledge base." It's the single most common ingredient in real-world agent projects — and a centerpiece of the resume project in Part 2.

Part 1B — Agents, the Loop, Memory & Multi-Agent

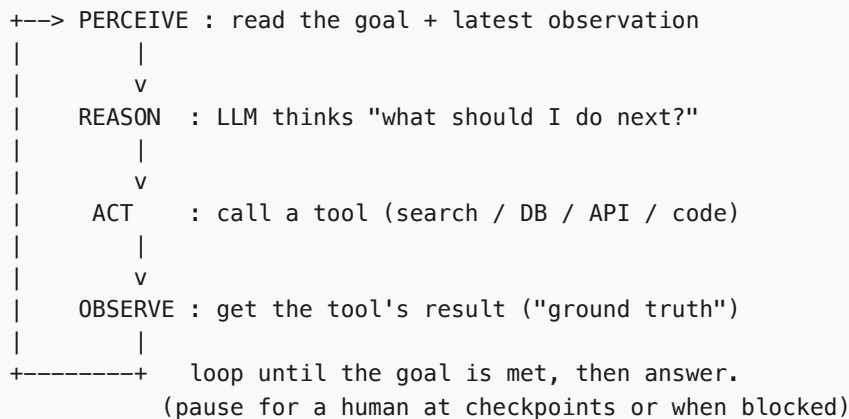
6. What is "Agentic AI" / an AI Agent?

In one sentence. An AI agent is an LLM placed inside a **loop** and given **tools**, so it can decide what to do, take an action, look at the result, and keep going until it reaches a goal — instead of just replying once.

The analogy. A plain LLM is like asking a knowledgeable friend a question and getting one answer. An agent is like hiring an assistant: you give them a goal ("book me a dentist appointment"), and they *independently* look things up, make calls, handle surprises, and come back when it's done.

The key distinction (this is the industry-standard framing, from Anthropic). - **Workflow** = you, the developer, write fixed steps and the LLM fills in blanks along a **predefined path**. Predictable, easy to control. - **Agent** = the **LLM itself decides** the steps dynamically, choosing tools and looping as needed. Flexible, handles open-ended problems. - **The advice:** start with the *simplest* thing that works (a single prompt, or a workflow). Only reach for a full autonomous agent when the task is **open-ended** and you genuinely **can't predict the number of steps** in advance. (Saying this in an interview shows maturity — agents are powerful but harder to control and more expensive.)

How an agent works — the loop:



The word "ground truth" matters: after each action the agent gets **real feedback from the world** (a search result, a database row, an error message) and re-plans based on it. That feedback loop is what makes agents robust.

Why it matters. This loop *is* agentic AI. Everything else (RAG, MCP, memory) is either a tool the agent calls or a support system for the loop.

7. The ReAct Pattern (Reason + Act)

In one sentence. ReAct is the foundational recipe for the agent loop: make the LLM **alternate between writing its reasoning and taking an action**, so thinking and doing reinforce each other.

The analogy. Thinking out loud while solving a problem: "I need the capital of France... let me look it up... okay, it's Paris... now I can answer." Writing the thought *before* the action keeps you on track.

How it actually works. The prompt nudges the LLM to produce a repeating pattern:

```
Thought: I need current weather; I should call the weather tool.  
Action: get_weather(city="Paris")  
Observation: 14°C, light rain.          <- tool result comes back  
Thought: I have what I need; I can answer now.  
Answer: It's 14°C and lightly raining in Paris.
```

This interleaving of **Thought** → **Action** → **Observation** (proposed in the ReAct paper, Yao et al., 2023) measurably **reduces hallucination** versus reasoning alone, because the model keeps checking itself against real tool results.

Why it matters. When you build an agent with LangGraph (Section 11), you're essentially implementing a ReAct-style loop. Knowing the name and the Thought/Action/Observation cycle is core agent vocabulary.

8. Tools / Function Calling

In one sentence. A "tool" is any function the agent is allowed to call — search the web, query a database, run code, hit an API — and "function calling" is the mechanism by which the LLM says *which* tool to run and *with what arguments*.

The analogy. Giving your assistant a set of labeled buttons: [Search] [Calculator] [Send Email]. The LLM reads the labels (tool descriptions) and presses the right button with the right input.

How it actually works. - You describe each tool to the LLM: its **name**, what it **does**, and its **inputs** (e.g., `get_weather(city: string)`). - When the LLM wants to use it, it outputs a structured request like `{"tool": "get_weather", "args": {"city": "Paris"}}`. - **Your code** actually runs the function and feeds the result back as the "observation." (The LLM never runs anything itself — it just *asks*; your program executes.)

Example. A calculator tool prevents the classic LLM weakness at arithmetic:

```
User: "What's 4839 * 2917?"  
LLM (reasons): I'm bad at big multiplication -> use the calculator tool.  
Action: multiply(4839, 2917)  
Observation: 14,115,363  
LLM: "4839 * 2917 = 14,115,363." (correct, because a real calculator ran)
```

Why it matters. Tools are the agent's **hands**. RAG-search, database lookups, sending a Slack message — all are tools. MCP (next) is a universal standard for providing them.

9. MCP (Model Context Protocol)

In one sentence. MCP is an open standard — a universal plug — that lets any AI agent connect to any external tool or data source without custom, one-off integration code.

The analogy. USB-C for AI. Before USB-C, every device had its own charger. MCP is the single connector: build a tool once as an "MCP server," and *any* MCP-compatible agent can use it — no bespoke glue per combination.

The problem it solves. Without a standard, connecting M agents to N tools means writing M×N custom integrations (an "N×M problem"). MCP turns that into M+N: each tool exposes itself once, each agent speaks MCP once.

How it actually works — client/server:

```
[ AI app = MCP CLIENT ] <----- MCP protocol -----> [ MCP SERVER ]
  (your agent)                                           exposes:
                                                         - tools (functions)
                                                         - resources (data)
                                                         - prompts

Example servers: a GitHub server, a Postgres server, a filesystem server,
your own "company-docs" server. The agent connects and can now use them all.
```

Who made it / freshness flag. Created by **Anthropic** (announced Nov 2024). In **Dec 2025**, governance was donated to the **Agentic AI Foundation** (a Linux-Foundation fund, co-founded with Block and OpenAI) — so today it's a **vendor-neutral industry standard**, not an Anthropic-only thing. Say it that way and you sound current.

Why it matters for your resume. MCP is one of the hottest terms in agentic AI right now. "Exposed the knowledge base and internal APIs as MCP servers so the agent could call them through a standard protocol" is a strong, current-sounding resume line.

10. Memory — Short-Term vs Long-Term

In one sentence. Because an LLM forgets everything between calls, agents bolt on memory: **short-term** (what's happening in this conversation) and **long-term** (facts that persist across sessions).

The analogy. Short-term memory is the **notepad on your desk** during one meeting; long-term memory is the **filing cabinet** you keep between meetings.

Short-term memory (a.k.a. working / context memory). - What it is: the recent conversation turns, kept so the agent stays coherent within a chat. - How it's implemented: the app keeps the running message history and re-sends it in the prompt each turn. Because the context window (Section 2) is finite, once the chat gets long you must **trim** old turns or **summarize** them ("earlier, the user said X..."). - Example: you say "my name is Vipin," and three turns later it still calls you Vipin — because those turns are still in the context being re-sent.

Long-term memory. - What it is: information that survives *across* sessions — user preferences, past decisions, learned facts. - How it's implemented: you **save** important snippets, **embed** them (Section 3), and store the vectors in a **vector database**. Next time, you **retrieve** the relevant memories by semantic similarity and paste them into the prompt. (Yes — long-term memory is basically RAG applied to the agent's own past.) - Example: last week you told the assistant "I prefer answers in bullet points." That preference was embedded and stored; today it's retrieved and the agent replies in bullets without being reminded.

```
SHORT-TERM: [ this chat's messages ] --re-sent each turn--> LLM
LONG-TERM:  save fact -> embed -> vector DB
           later: question -> retrieve similar memories -> add to prompt
```

Why it matters. Memory is what makes an agent feel *persistent and personal* rather than amnesiac. It's also a great resume talking point because it shows you understand the context- window limitation and how to engineer around it.

11. Multi-Agent Systems (agent-to-agent)

In one sentence. Instead of one agent doing everything, you use **several specialized agents** that hand work to each other — like a team with distinct roles.

The analogy. A company. A generalist doing sales, engineering, and accounting badly is worse than a small team where each person is a specialist and a manager coordinates them.

Common orchestration patterns.

```
SUPERVISOR (most common):
  [ Supervisor agent ]
    /      |      \
  [Researcher][Coder][Writer]    supervisor routes each subtask to the
                                   right specialist, collects results.

PIPELINE / SEQUENTIAL:
  [Agent A] -> [Agent B] -> [Agent C]    output of one feeds the next
                                           (e.g., research -> draft -> edit)

(Agents "communicate" by passing messages/structured data to each other,
 often coordinated by a graph – see LangGraph, next section.)
```

Example. A "research assistant" system: a **Supervisor** receives "write me a market brief on X." It asks a **Researcher agent** (which uses RAG + web search) to gather facts, passes those to a **Writer agent** to draft the brief, then a **Critic agent** to check it. Each is a focused agent with its own tools and prompt.

Why it matters. Multi-agent is where the field is heading, and it makes an impressive (but still buildable) resume project. Caution to voice in interviews: **more agents = more complexity and cost**, so use multi-agent only when a single agent genuinely struggles — the same "start simple" wisdom from Section 6.

Part 1C — The Tooling (LangChain, LangGraph, FastAPI, Vector DBs) & Key Decisions

12. LangChain

In one sentence. LangChain is a Python (and JavaScript) toolkit that gives you ready-made building blocks for LLM apps — connecting to models, loading and chunking documents, building RAG, defining tools — so you don't write all the plumbing yourself.

The analogy. A kitchen with pre-prepped ingredients and appliances. You *could* mill your own flour, but LangChain hands you the mixer, the measured ingredients, and recipes so you can cook faster.

What it gives you (the useful pieces). - **Model wrappers:** one consistent way to call Claude, GPT, Gemini, local models — swap providers by changing one line. - **Document loaders & splitters:** read PDFs/websites and chop them into chunks for RAG. - **Vector store integrations:** talk to Chroma, Pinecone, pgvector, etc. - **Tool & prompt abstractions:** define tools and prompt templates cleanly.

Example. Loading docs and embedding them for RAG is a few lines instead of a few hundred:

```
# conceptual — check current docs for exact names
docs = PyPDFLoader("handbook.pdf").load()
chunks = RecursiveCharacterTextSplitter(chunk_size=500).split_documents(docs)
store = Chroma.from_documents(chunks, OpenAIEmbeddings()) # embed + store
results = store.similarity_search("refund policy?") # semantic search
```

Why it matters. LangChain is the "batteries-included" layer for the *components* of an LLM app. For simple, linear flows (load → retrieve → answer) it's often all you need.

Freshness flag: LangChain's exact class/function names change between versions. Learn the *concepts* here; confirm the current API in the docs when you code.

13. LangGraph

In one sentence. LangGraph is a companion library (from the LangChain team) for building **agents as graphs** — you define nodes (steps) and edges (transitions), which is exactly what you need for the looping, branching, stateful behavior of real agents.

The analogy. LangChain is a straight recipe (do step 1, then 2, then 3). LangGraph is a **flowchart with loops and decision points**: "if the answer isn't good enough, go back and search again." Agents need flowcharts, not straight lines.

LangChain vs LangGraph — when to use which (a common interview question):

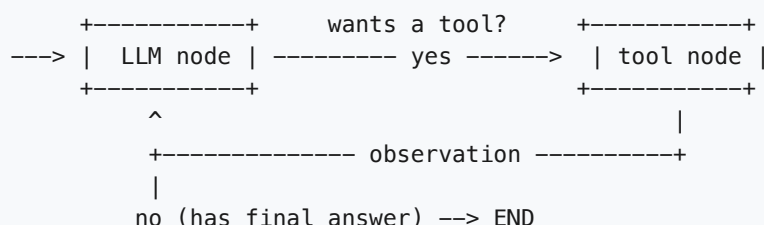
Use LANGCHAIN when the flow is simple & linear:
load docs -> retrieve -> answer. No looping, no branching.

Use LANGGRAPH when you need agentic control flow:

- loops (keep acting until the goal is met) <- the agent loop!
- branching ("if X do this, else that")
- persistent STATE passed between steps
- multiple agents coordinating (supervisor, pipeline)
- human-in-the-loop pauses / checkpoints

They're **complementary**, not competitors: you typically use LangChain's components (loaders, model wrappers, tools) *inside* LangGraph's graph structure.

How it actually works (the mental model). - You define a **state** (a shared object holding the conversation, retrieved docs, scratch notes). - You define **nodes** — each node is a function that reads the state and updates it (e.g., a "retrieve" node, a "call LLM" node, a "call tool" node). - You define **edges** — including **conditional edges** ("after the LLM node, if it requested a tool, go to the tool node; otherwise, finish"). That conditional edge back to the LLM is what creates the **agent loop**.



Why it matters for your resume. "Orchestrated the agent's reason-act-observe loop and multi-agent routing with **LangGraph**, using a shared state graph with conditional edges" is a precise, senior-sounding description of real agent engineering.

14. Vector Databases

In one sentence. A vector database stores embeddings (Section 3) and is optimized to answer "which stored items are most **similar in meaning** to this query?" extremely fast.

The analogy. A librarian who organizes books not alphabetically but **by topic proximity**, so "find me things like this one" is instant.

How it actually works. You store (vector, original_text, metadata) rows. At query time you embed the query and the DB returns the **nearest vectors** (by cosine similarity), using clever indexing (e.g., approximate nearest-neighbor) so it's fast even over millions of items.

Ones you'll hear about. **Chroma** (simple, great for learning/local), **pgvector** (adds vectors to PostgreSQL — nice if you already use Postgres), **Pinecone** / **Weaviate** / **Qdrant** / **Milvus** (managed/production-scale).

Why it matters. It's the storage engine behind **both** RAG and long-term memory. In your project, "vector DB" = where the knowledge and the memories live.

15. FastAPI

In one sentence. FastAPI is a modern Python web framework used to wrap your agent in an **API** — a web address other programs (or a chat UI) can send requests to and get answers back.

The analogy. Your agent is a chef in a kitchen. FastAPI is the **service window**: customers (apps, front-ends) place orders through it and receive dishes, without walking into the kitchen.

How it actually works. - You define **endpoints** (URLs) like `POST /chat`. A function runs when that URL is hit. - FastAPI handles the web plumbing: parsing JSON requests, validating inputs (via type hints + Pydantic), returning JSON responses, and **auto-generating interactive API docs**. - It's **async-friendly** and fast — well-suited to AI apps that spend time waiting on LLM calls, and it supports **streaming** responses (tokens appearing live, like ChatGPT typing).

Example.

```
# conceptual
from fastapi import FastAPI
app = FastAPI()

@app.post("/chat")
async def chat(body: ChatRequest):
    answer = agent.invoke(body.message) # run the LangGraph agent
    return {"answer": answer}
# now any app can POST to http://your-server/chat and get a reply
```

Why it matters. An agent that only runs in your terminal isn't a product. FastAPI is how you turn it into a **service** — the difference between "I ran a script" and "I built and deployed an agent API" on a resume.

16. Fine-tuning vs Prompting vs RAG (the key decision)

This is one of the **most important practical decisions** in applied AI, and a favorite interview question. All three change what the model produces — but very differently.

Prompting — *just ask well*. - What: craft clear instructions/examples in the prompt. No training, no infrastructure. - The analogy: giving a skilled worker better instructions. - Use when: the base model already knows enough; you just need to steer format/tone/task. - Cost: cheapest and fastest. **Always try this first.**

RAG — *give it the right facts at answer time*. - What: retrieve relevant documents and inject them into the prompt (Section 5). - The analogy: an open-book exam — the knowledge comes from the book, not the student's memory. - Use when: the model needs **private, current, or factual** information it wasn't trained on, and you want to **reduce hallucination** and cite sources. - Cost: moderate; you maintain a vector DB. Facts update instantly by updating documents (no retraining).

Fine-tuning — *retrain the model on your data to change its behavior*. - What: continue training the model on many examples so it internalizes a **style, format, or skill** (e.g., always output legal-contract language, or a specific JSON schema). - The analogy: sending the worker to a training course to build a new habit. - Use when: you need a consistent **behavior/style/format** that prompting can't reliably get, and you

have many high-quality examples. **Not** the right tool for injecting facts (RAG is). - Cost: highest — needs data prep, compute, and re-doing it when things change.

Decision cheat-sheet:

Need better instructions/format? → PROMPTING (try first)

Need private/current FACTS, no lies? → RAG

Need a consistent SKILL/STYLE/format? → FINE-TUNING

(These combine! Most real systems = prompting + RAG; fine-tuning only when the first two aren't enough.)

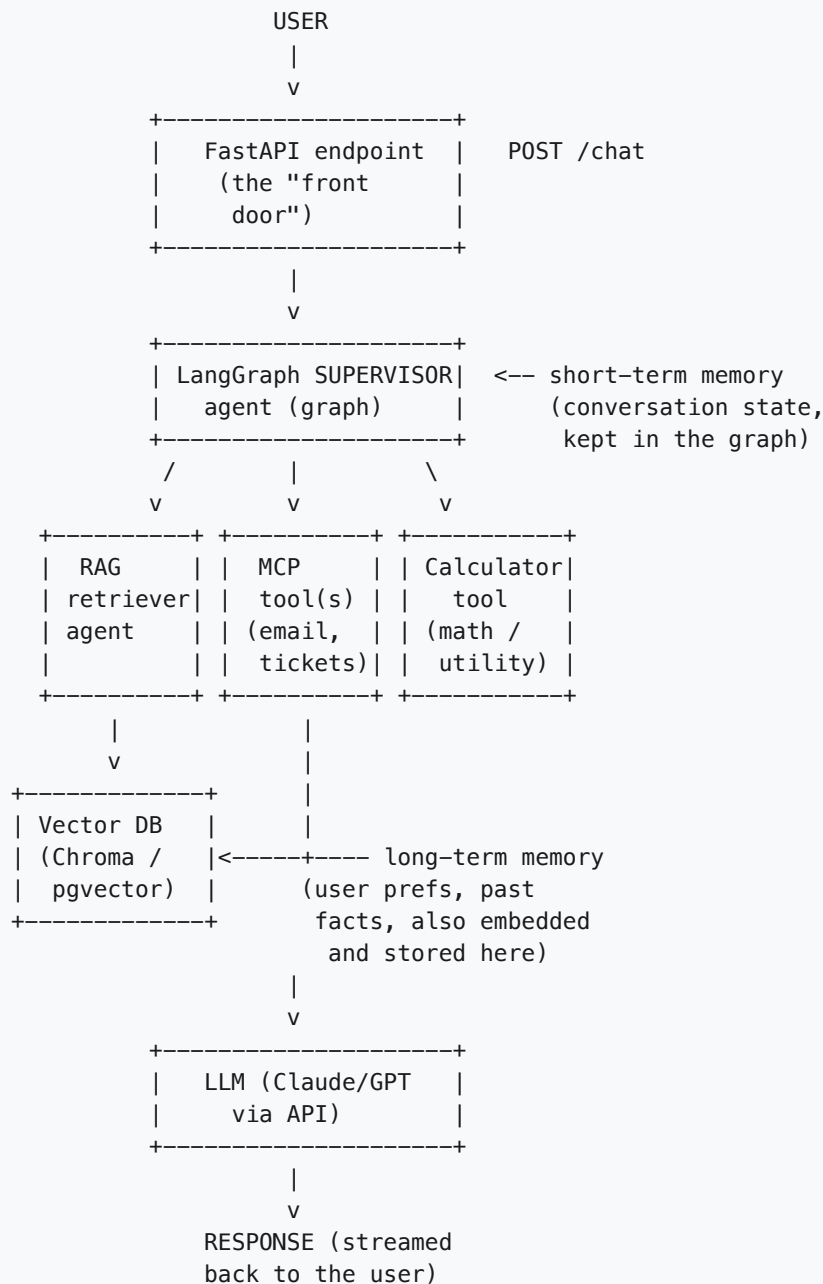
The one-liner to remember: *Prompting changes what you ask; RAG changes what it knows; fine-tuning changes how it behaves.* Facts → RAG. Behavior → fine-tuning. Say this and you sound like you've actually shipped something.

Part 2 — The Resume Project: Multi-Agent Knowledge Assistant

What you're building (in one paragraph)

You're going to build a **Multi-Agent Knowledge & Support Assistant** — an API-backed AI assistant that can answer questions about a real body of documents (think: a company's internal handbook, a product's user manual, or a set of policy PDFs), can *act* by calling tools (look something up, do a calculation, send an email, check a live system), remembers both the current conversation and things it learned about you across sessions, and is served as a proper web API rather than a script you run in a terminal. Picture a support team lead who's tired of answering "what's our refund policy?" fifty times a day, or a new engineer who wants a bot that can answer "how does our deploy pipeline work?" from the team's wiki instead of pinging five people on Slack — that's who'd use this. It's impressive on a resume because it touches nearly every concept from Part 1 (RAG, agents, tools, MCP, memory, multi-agent, LangGraph, FastAPI) in one coherent system, and it's doable because each piece is a well-trodden, well-documented pattern — you're assembling known building blocks, not inventing new ones.

The architecture (big picture)



Walking through this step by step:

1. **The user** sends a message — a question, a request, whatever — to your service. They don't know or care that agents and vector databases exist underneath; they just see a chat box or an API response.
2. **FastAPI** is the front door. It receives the HTTP request, validates the JSON body (is there actually a `message` field?), and hands the message to your agent. This is the only part of the system exposed to the outside world.
3. **The LangGraph supervisor** is the "brain" that decides what to do next. It holds the **state** of the conversation (that's your short-term memory — the running list of messages) and, on each turn, asks the LLM: "given this message and the tools available, what should happen next?"

4. Based on the LLM's decision, the supervisor **routes** the request to the right specialist: the **RAG retriever** if it's a knowledge question, an **MCP tool** if it needs to reach an external system (send an email, look something up on the web, file a ticket), or a **calculator** if it's arithmetic the LLM shouldn't attempt by itself.
5. The **RAG retriever** embeds the question and searches the **vector database** for the most relevant chunks of your documents — this is exactly the RAG pattern from Part 1, giving the LLM real facts instead of letting it guess.
6. The same **vector database** doubles as **long-term memory**: facts and preferences you want the agent to remember across sessions (e.g. "this user prefers short answers") are embedded and stored there too, and retrieved the same way — by semantic similarity.
7. Whatever came back from the specialist (retrieved text, a tool result, a calculation) is fed to the **LLM** as an "observation," and the LLM reasons over it to produce the final answer — this is the ReAct loop from Part 1, running inside the LangGraph graph.
8. The answer flows back up through the supervisor and FastAPI, ideally **streamed** token-by-token so the user sees it appear live, the way ChatGPT does.

The tech stack (and WHY each choice)

Component	Technology	Why
Language	Python	The de facto language for AI tooling — every library in this table has a mature Python SDK first.
LLM	Claude or GPT, via API	You don't train or host a model; you call a hosted one over an API and get frontier-level reasoning for the cost of a few cents per request.
Orchestration	LangGraph	Gives you the graph, state, loops, and conditional edges needed to implement the agent loop and multi-agent routing — plain function calls can't express "loop until done" as cleanly.
Components	LangChain	Supplies the plumbing — document loaders, text splitters, model wrappers, vector-store connectors — so you're not hand-rolling PDF parsing or chunking logic.
Vector DB	Chroma (local/dev) or pgvector / Pinecone (prod)	Chroma runs embedded with zero setup, perfect for learning and demos; pgvector (if you already run Postgres) or Pinecone (managed, scales) are the realistic production upgrade path.
Embeddings	An embedding model (e.g. an OpenAI or Voyage embeddings endpoint, or a local sentence-transformers model)	Turns your documents and queries into vectors so "similar meaning" becomes "nearby points" — the whole basis of RAG and long-term memory.
API layer	FastAPI	Turns your agent from a script into a service other programs can call, with automatic input validation and built-in support for streaming responses.

Component	Technology	Why
Tool connectivity	MCP servers	Instead of writing bespoke glue code per external system, you expose (or reuse) each tool — email, ticketing, filesystem, a database — as an MCP server your agent can plug into, the "USB-C for AI" idea from Part 1.
Memory	Short-term: in LangGraph's state (in-memory or checkpointed) · Long-term: in the vector DB	Short-term needs to be fast and scoped to one conversation; long-term needs to persist and be searchable by meaning — two different jobs, two different mechanisms.
Optional UI	Streamlit (fastest) or a small React front-end	Streamlit gets you a usable chat UI in under an hour with pure Python; React is the better resume line if you want to show front-end skills too, but it's optional — the API is the real deliverable.

How a request flows end to end

Let's trace one concrete, real question through the whole system:

User: "What's our refund policy, and email me a summary?"

1. USER -> FastAPI
POST /chat {"message": "What's our refund policy, and email me a summary?", "user_id": "vipin"}
2. FastAPI -> LangGraph supervisor
Loads short-term memory (this chat's history) and passes the new message into the graph's state.
3. Supervisor -> LLM (Thought)
"Thought: this needs two things – a fact from our docs, and an action (sending an email). I should retrieve first, then act."
4. Supervisor -> RAG retriever agent (Action)
"Action: retrieve('refund policy')"
-> embeds the query -> searches the vector DB
-> Observation: 3 chunks come back, e.g. "Refunds are accepted within 30 days of purchase with a receipt..."
5. Supervisor -> LLM (Thought, with the observation in hand)
"Thought: I now know the policy. Next I need to email a summary to the user – I'll use the email tool."
6. Supervisor -> MCP email tool (Action)
"Action: send_email(to='vipin@company.com', subject='Refund Policy Summary', body='<generated summary>')"
-> the MCP server actually sends the email
-> Observation: {"status": "sent", "message_id": "..."}
7. Supervisor -> LLM (Thought)
"Thought: retrieval succeeded, email sent – I have everything needed to answer the user now."
8. LLM -> final answer, streamed back through FastAPI:
"Our refund policy allows returns within 30 days of purchase with a receipt. I've emailed you a summary at vipin@company.com."
9. Supervisor -> memory update
 - Short-term: this exchange is appended to the conversation state for the next turn.
 - Long-term: if something worth remembering came up (e.g. "user prefers email summaries over long chat replies"), it gets embedded and written to the vector DB for future sessions.
10. FastAPI -> USER
The streamed response finishes rendering in the chat UI (or is returned as JSON if called headlessly).

Notice steps 3–7 are a textbook **Thought -> Action -> Observation** ReAct cycle repeated twice — once for retrieval, once for the email tool — and step 9 is exactly the short-term/long-term memory split from Part 1 in action, not just in theory.

Build it in phases

Don't build all of this at once. Build it in phases, and each phase is already a working, demoable thing on its own.

Phase 1 — Basic RAG Q&A over docs + FastAPI endpoint. Load a handful of PDFs or markdown docs, chunk them, embed them into Chroma, and write a single FastAPI endpoint that: takes a question, retrieves the top chunks, stuffs them into a prompt, and returns the LLM's answer. No agent yet — this is plain RAG behind an API. *Demonstrates:* embeddings, vector search, RAG, FastAPI.

Phase 2 — Make it an agent with tools via LangGraph + the ReAct loop. Wrap the RAG step as a *tool* rather than a hardcoded call, add a calculator tool, and put an LLM node in front that decides which tool (if any) to call, looping via a conditional edge until it has an answer. *Demonstrates:* agent loop, ReAct, tools/function calling, LangGraph.

Phase 3 — Add memory: short-term + long-term. Keep the running conversation in the graph's state so multi-turn chats stay coherent (short-term). Add a second vector-DB collection for "memories" — save notable facts/preferences, embed them, and retrieve relevant ones each turn to inject into the prompt (long-term). *Demonstrates:* context-window limits, short vs long-term memory, RAG applied to the agent's own history.

Phase 4 — Add MCP servers for tools. Swap at least one hand-rolled tool (e.g. "send an email," "search the web") for a real or self-written **MCP server**, and connect your agent to it as an MCP client instead of calling a Python function directly. *Demonstrates:* MCP, tool standardization, real integration experience.

Phase 5 — Go multi-agent: supervisor + specialists. Split the single agent into a **supervisor** plus focused specialists — e.g. a Retriever agent (RAG only), a Support-Actions agent (MCP tools only), maybe a Critic agent that double-checks answers against retrieved sources before they go out. The supervisor routes each request and combines results. *Demonstrates:* multi-agent orchestration, the supervisor pattern.

Phase 6 — Polish: streaming, a UI, evaluation. Turn on token streaming in FastAPI so answers appear live. Add a Streamlit (or small React) chat UI on top of the API. Write a small evaluation set — a list of questions with known-good answers or sources — and script a check for how often the agent retrieves the right chunk and avoids hallucinating. *Demonstrates:* production polish, the kind of evaluation rigor interviewers actually ask about.

Resume bullet points (honest, strong)

Project title: *Multi-Agent Knowledge & Support Assistant — a retrieval-grounded, tool-using AI agent served as a production-style API.*

- Designed and built a multi-agent AI assistant using **LangGraph** and **LangChain**, orchestrating a supervisor agent that routes requests to specialist agents for retrieval, tool use, and response generation.
- Implemented a **RAG pipeline** (chunking, embeddings, vector search over Chroma/pgvector) to ground answers in a real document set, reducing hallucinations by anchoring responses in retrieved context instead of model recall alone [measured hallucination/error rate reduced by X%].

- Exposed internal tools (email, ticketing, lookups) to the agent via the **Model Context Protocol (MCP)**, replacing bespoke integration code with a standard, reusable tool-serving pattern.
 - Engineered a two-tier memory system — in-graph short-term conversation state plus a vector-store-backed long-term memory — enabling the agent to retain user preferences and context across sessions.
 - Served the agent as a **FastAPI** service with streaming responses, turning a local prototype into a callable API consumable by other applications or a chat UI, and handled [X] requests/day in local load testing.
 - Built an evaluation harness to score retrieval accuracy and answer quality against a hand-labeled question set, catching regressions before they reached users [improved retrieval accuracy by X%].
-

Interview talking points

"Why LangGraph over plain LangChain?" Plain LangChain is great for a linear pipeline — load, retrieve, answer — but this project needs loops (keep acting until the goal is met), branching (which specialist handles this?), and shared state across steps. LangGraph models that as a graph with conditional edges, which is a more honest representation of what an agent actually does than a straight-line chain.

"How did you prevent hallucinations?" Mainly through RAG: the agent answers from retrieved document chunks rather than from parametric memory, and I can point to which chunk backed a given answer. Beyond that, the ReAct-style loop forces the model to check its reasoning against real observations (retrieval results, tool outputs) before answering, and a Critic/verification step (if I added the multi-agent phase) cross-checks the draft answer against the sources before it's returned.

"Why MCP instead of just writing tool functions?" Hand-written tool functions work fine for a single project, but they don't generalize — every new agent needs its own glue code for the same external system. MCP standardizes the "tool boundary" so the same server (say, an email-sending server) can be reused by this agent or any other MCP-compatible agent without rewriting integration code. It's also simply where the industry is heading, so it was worth the practice.

"How does memory actually work here?" Short-term memory is just the running conversation kept in the LangGraph state and re-sent to the LLM each turn — it's forgotten once the session ends unless I explicitly persist it. Long-term memory is different: specific facts or preferences get embedded and written to a vector database, and on future turns (even in a new session) I retrieve the most relevant ones by similarity and add them to the prompt. It's essentially RAG, but over the agent's own memories instead of a document set.

"What would you improve, or how would you scale it?" I'd move the vector store from Chroma to pgvector or Pinecone for concurrent access and durability, add proper request-level tracing so I can see each agent's Thought/Action/Observation steps in production, add caching for repeated questions, and add guardrails (input validation, rate limiting, PII checks) before letting it touch real user data or send real emails. I'd also grow the evaluation set so regressions are caught automatically rather than by eyeballing answers.

"Prompting vs RAG vs fine-tuning — why did you choose RAG?" The core problem here is factual: the agent needs to answer from a specific, changeable set of documents it wasn't trained on, and I need to be able to update those documents without retraining anything. Prompting alone can't inject facts the

model doesn't already know, and fine-tuning is the wrong tool for facts — it's for teaching a consistent style or skill, and it's expensive to redo every time the docs change. RAG lets me update an answer's grounding just by updating the vector store, instantly, with no training run.

Honesty & ethics note

Only put this project on your resume if you actually build it. It's designed to be built in real phases, each one a genuine working milestone — build at least through Phase 3 or 4 before you call it a project, and be ready to open the code and walk an interviewer through any part of it, live, on request. If you include a metric (a percentage, a latency number, an accuracy score), it should be a number you actually measured on your own test set, not a plausible-sounding guess — leave the [X%] placeholders blank, or delete that clause, if you haven't measured it. Interviewers in this space increasingly ask pointed follow-up questions specifically because so many resumes overclaim on AI projects; a smaller project you deeply understand beats a bigger one you can't explain.
